

Chapter 4: ReAct Pattern (Reason + Act) and Agent Execution Loops

Part II: Agentic Architecture Patterns

Book: Agentic AI & RAG Systems Handbook

Version: 1.0.0

Chapter ID: 04

File Name: chapter-04-react-pattern.md

Chapter Overview

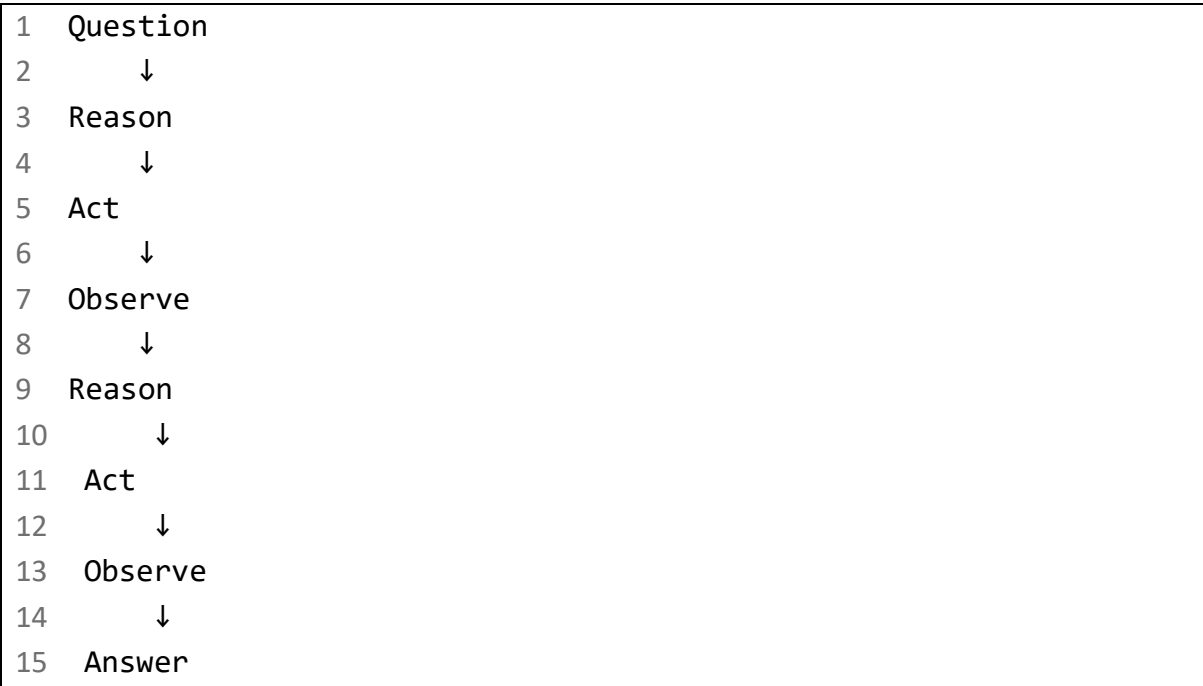
The ReAct (Reason + Act) pattern is the foundation of modern Agentic AI.

Before ReAct, most AI workflows followed a simple model:



The model either knew the answer or hallucinated one.

ReAct introduced a fundamentally different approach:



This transformed LLMs from passive text generators into active decision-making systems capable of:

- Tool use
- Evidence gathering
- Multi-step reasoning
- Dynamic planning
- Adaptive execution

Nearly every production agent architecture today derives from ReAct principles.

Learning Objectives

By the end of this chapter you should be able to:

- Understand the ReAct paradigm
 - Design ReAct execution loops
 - Build tool-using agents
 - Implement ReAct in LangGraph
 - Compare ReAct to alternative patterns
 - Govern ReAct systems securely
 - Evaluate ReAct performance
 - Recognize production anti-patterns
 - Discuss ReAct architecture in Staff/Principal interviews
-

Prerequisites

Required:

- Chapter 1: Generative AI Foundations
 - Chapter 2: Agentic AI Fundamentals
 - Chapter 3: Agent Design Principles
-

1. Introduction

Traditional LLM execution looks like:

1 Question

```
2      ↓
3  Model
4      ↓
5  Response
```

But many enterprise tasks require:

```
1  Research
2  Investigation
3  Validation
4  Retrieval
5  Tool Usage
6  Decision Making
```

These cannot be solved reliably through generation alone.

Example:

```
1  Investigate customer outage.
```

The model must:

```
1  Search incidents
2      ↓
3  Read logs
4      ↓
5  Analyze events
6      ↓
7  Validate evidence
8      ↓
9  Recommend action
```

This is exactly what ReAct enables.

2. Core Concepts

What Is ReAct?

ReAct stands for:

```
1  RE
2  Reasoning
3
4  ACT
5  Action
```

The model alternates between:

- | | |
|---|---------|
| 1 | Think |
| 2 | Do |
| 3 | Observe |
| 4 | Think |
| 5 | Do |
| 6 | Observe |

until sufficient evidence exists to produce a final answer.

Core Principle

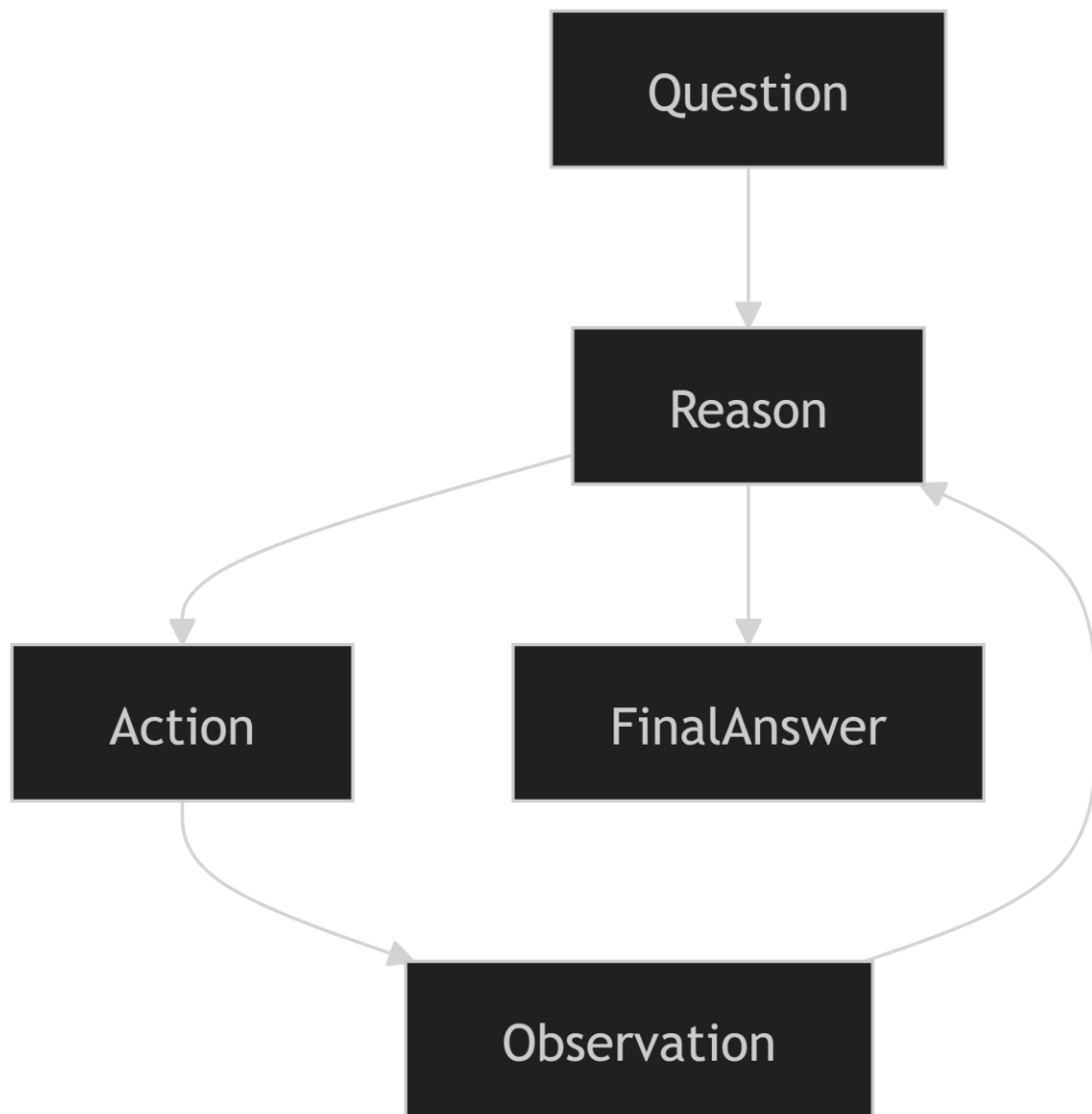
The agent should not:

- | | |
|---|--------|
| 1 | Assume |
|---|--------|

when it can:

- | | |
|---|--------|
| 1 | Verify |
|---|--------|
-

ReAct Cycle



3. Why ReAct Matters

Without ReAct:

- | | |
|---|----------|
| 1 | Question |
| 2 | ↓ |
| 3 | Guess |

With ReAct:

- | | |
|---|----------|
| 1 | Question |
|---|----------|

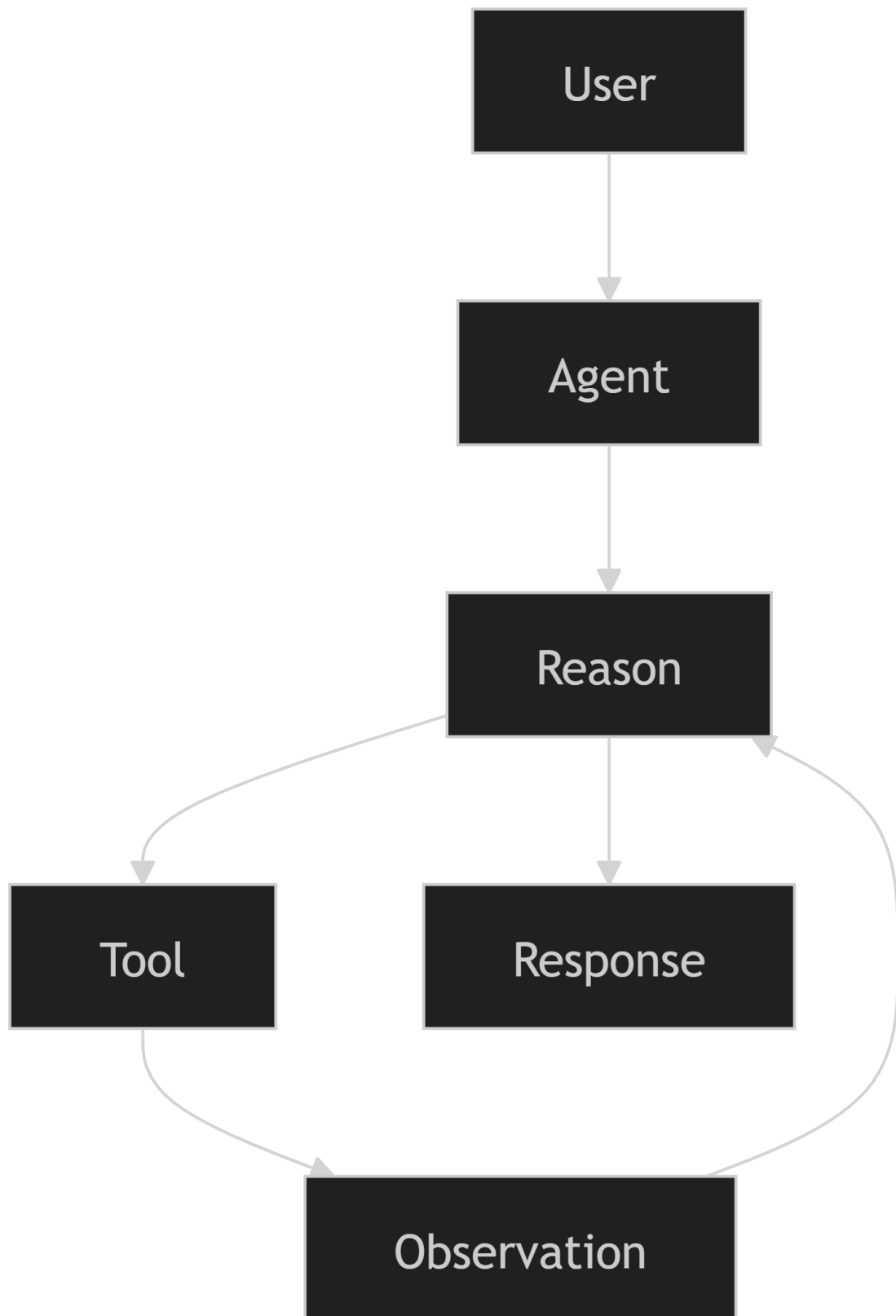
2	↓
3	Investigate
4	↓
5	Validate
6	↓
7	Respond

Benefits include:

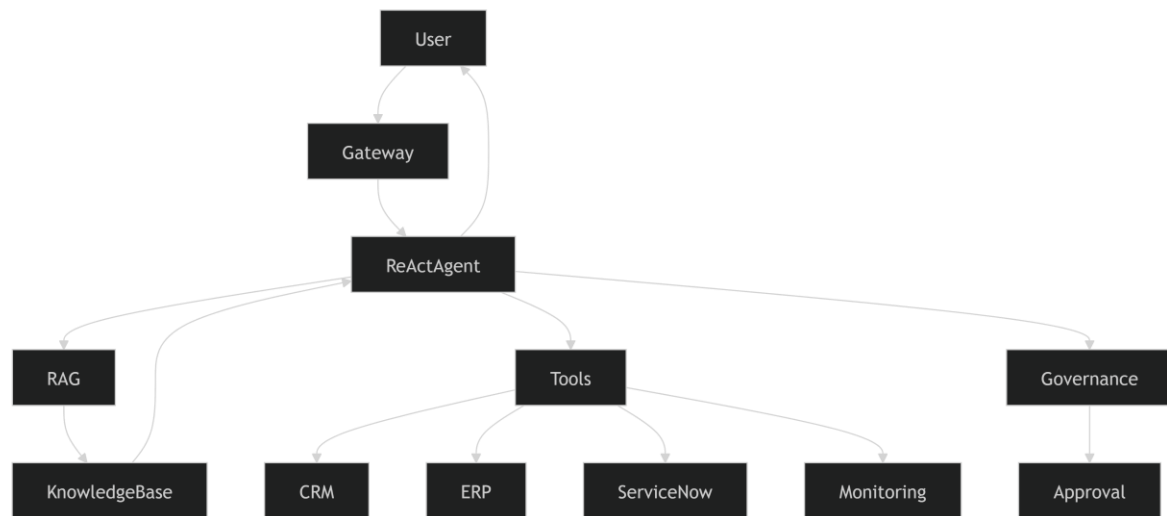
- Higher factual accuracy
 - Better grounding
 - Tool utilization
 - Reduced hallucinations
 - Improved explainability
-

4. ReAct Architecture

Basic Architecture



Enterprise Architecture



5. Detailed Technical Deep Dive

Step 1: Reason

Agent asks:

1 What information do I need?

Example:

1 Need customer outage records.

Step 2: Act

Tool invocation occurs.

1 Search Incidents

Tool:

1 search_incidents(customer_id)

Step 3: Observe

Receive results.

1 Found outage yesterday.

Step 4: Reason Again

Determine next step.

1 Need infrastructure logs.

Step 5: Repeat

Until either:

- Goal achieved
 - Escalation required
 - Iteration limit reached
-

Step 6: Final Response

Synthesize findings.

6. Simple ReAct Example

User Request

1 Why was the payment service unavailable?
--

Reason

1 Need outage history.

Action

1 Search incidents.

Observation

1 Database outage at 10:14 AM.

Reason

1 Need root cause.

Action

1 Query monitoring.

Observation

1 Connection pool exhausted.

Reason

1 Enough evidence gathered.

Answer

1 Payment service failed due to database connection pool exhaustion.
--

7. Architecture Patterns within ReAct

Pattern 1: Search Then Reason

1 Search
2 ↓
3 Observe
4 ↓
5 Reason

Best for:

- Knowledge systems
 - Enterprise search
 - Research
-

Pattern 2: Database Then Reason

1 Query
2 ↓
3 Reason

Best for:

- Analytics

- Reporting
 - Operational intelligence
-

Pattern 3: Multi-Tool ReAct

1	Search
2	↓
3	Logs
4	↓
5	Metrics
6	↓
7	Reason

Best for:

- Incident response
 - RCA
 - Compliance
-

Pattern 4: Corrective ReAct

1	Retrieve
2	↓
3	Evaluate
4	↓
5	Rewrite
6	↓
7	Retrieve Again

Foundation for Corrective RAG.

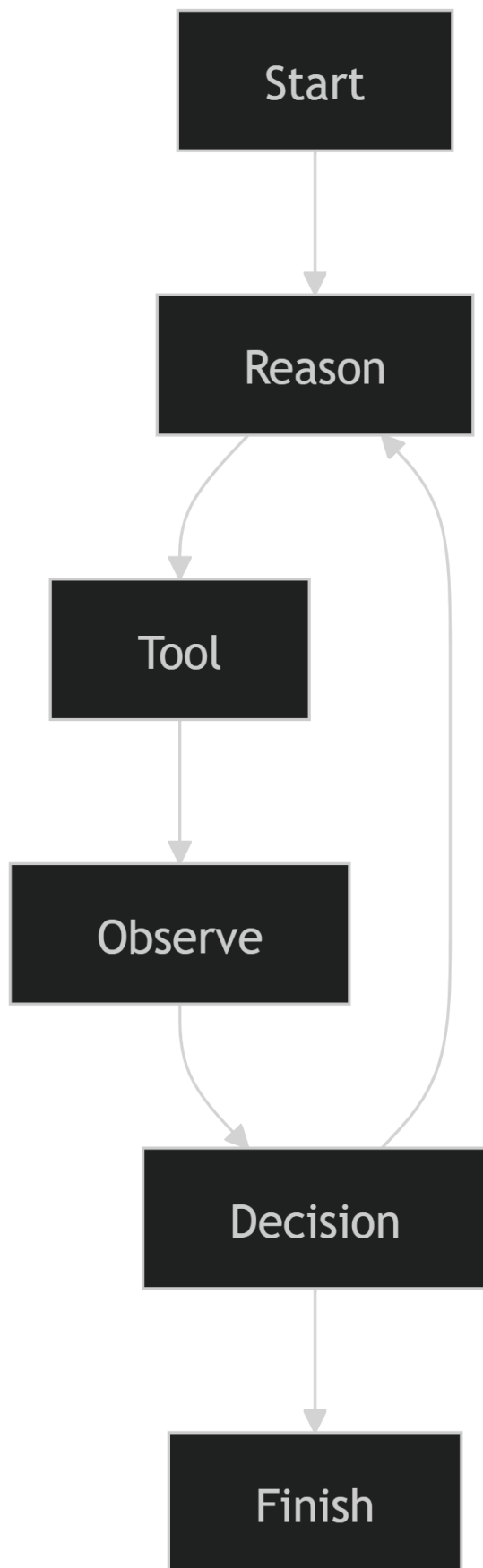
Pattern 5: Governed ReAct

1	Reason
2	↓
3	Policy
4	↓
5	Tool

Required for enterprise use.

8. LangGraph ReAct Implementation

Graph Structure



State Model

```
1 from typing import TypedDict
2
3 class ReActState(TypedDict):
4     question: str
5     thoughts: list
6     observations: list
7     answer: str
```

Reason Node

```
1 def reason(state):
2     return {
3         "thoughts": [
4             "Need more evidence"
5         ]
6     }
```

Tool Node

```
1 def execute_tool(state):
2     return {
3         "observations": [
4             "Incident found"
5         ]
6     }
```

Decision Node

```
1 def decide(state):
2     if sufficient_evidence(state):
3         return "finish"
4
5     return "reason"
```

9. CrewAI ReAct Example

```
1 researcher = Agent(  
2     role="Research Analyst",  
3     goal="Collect evidence"  
4 )
```

Execution naturally follows:

```
1 Reason  
2 ↓  
3 Tool  
4 ↓  
5 Observe
```

10. AutoGen ReAct Example

AutoGen agents frequently perform iterative reasoning loops through conversations.

```
1 Agent  
2 ↓  
3 Tool  
4 ↓  
5 Result  
6 ↓  
7 Agent
```

This is a ReAct variant.

11. Semantic Kernel ReAct Example

Semantic Kernel planners commonly execute:

```
1 Goal  
2 ↓  
3 Skill Selection  
4 ↓  
5 Execution  
6 ↓  
7 Observation
```

which aligns closely with ReAct principles.

12. OpenAI Tool Calling Example

```
1  tools = [  
2      {  
3          "type": "function",  
4          "function": {  
5              "name": "search_incidents"  
6          }  
7      }  
8  ]
```

Model reasons.

Model selects tool.

Tool executes.

Observation returns.

Reasoning continues.

13. Anthropic Tool Use

Anthropic's tool-use architecture is similarly ReAct-oriented.

```
1  Think  
2  ↓  
3  Use Tool  
4  ↓  
5  Receive Result  
6  ↓  
7  Think Again
```

14. Production Example

IT Incident Agent

Goal:

```
1  Investigate outage.
```

Execution:

```
1  Logs
2  ↓
3  Metrics
4  ↓
5  Incidents
6  ↓
7  Knowledge Base
8  ↓
9  Reasoning
10 ↓
11 Recommendation
```

Tools:

- Splunk
 - Datadog
 - ServiceNow
 - RAG system
-

15. Cloud Implementations

AWS

Tools:

- Bedrock
- Lambda
- OpenSearch
- CloudWatch

Flow:

```
1  Agent
2  ↓
3  CloudWatch
4  ↓
5  OpenSearch
6  ↓
7  Reasoning
```

Azure

Tools:

- Azure OpenAI
 - Azure AI Search
 - Application Insights
-

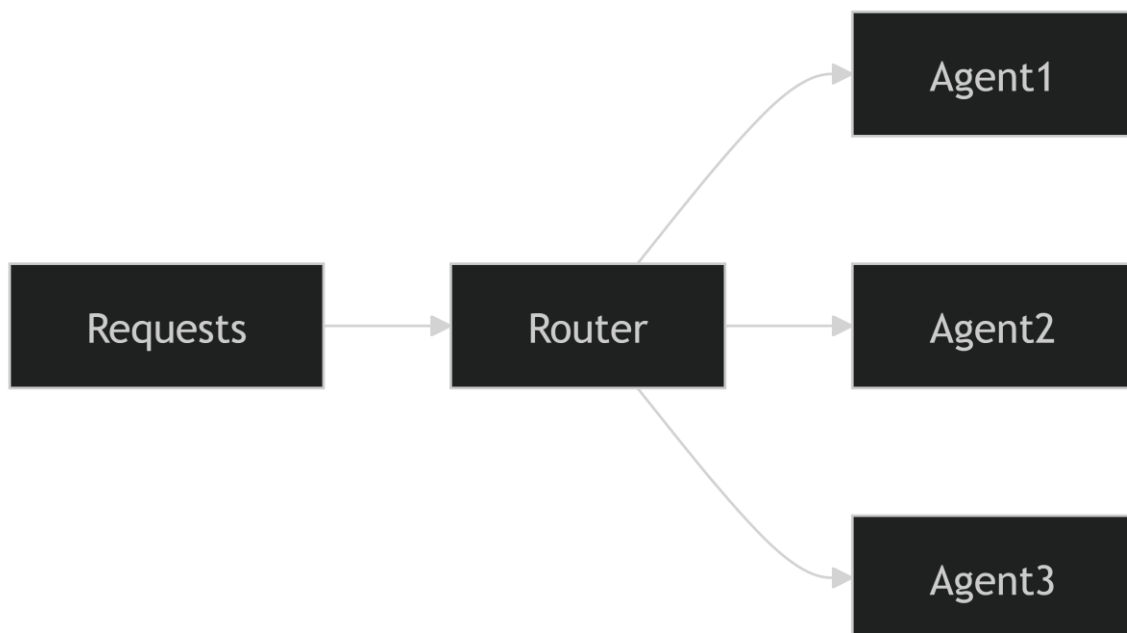
GCP

Tools:

- Vertex AI
 - BigQuery
 - Cloud Monitoring
-

16. Scalability

Common scaling model:



Scale ReAct agents horizontally.

17. Reliability

Controls:

- Retry limits
- Timeouts
- Tool fallbacks
- Checkpoints
- Human escalation

Example:

```
1 max_iterations: 10
2 timeout_seconds: 300
```

18. Security

Major risks:

Prompt Injection

```
1 Ignore previous instructions.
```

Tool Abuse

```
1 Delete records.
```

Excessive Permissions

Agent has too many tools.

Enterprise Control

Reason



Policy



Validation



Tool

19. Observability

Track:

- Iterations
- Tool calls
- Errors
- Cost
- Latency
- Success rate

Example metrics:

1	Average_iterations
2	Tool_success_rate
3	Cost_per_task

20. Trade-Offs

Benefit	Cost
Better accuracy	Higher latency
Better grounding	More tool calls
Better reasoning	More tokens
Greater flexibility	More complexity

21. Common Mistakes

Mistake 1

ReAct for every use case.

Mistake 2

Unlimited loops.

Mistake 3

No iteration limits.

Mistake 4

No tool governance.

Mistake 5

No evidence validation.

22. Anti-Patterns

ReAct Everywhere

Not every workflow needs reasoning.

Tool Explosion

Agent receives hundreds of tools.

Selection quality drops.

Hidden Tool Chains

Impossible to debug.

Autonomous Production Actions

High-risk operations without validation.

23. Architecture Decision Matrix

Scenario	ReAct Fit
Enterprise Search	Excellent
Incident Investigation	Excellent
Customer Support	Excellent
Compliance	Excellent
Fixed Workflow	Poor

Static FAQ	Poor
Deterministic Process	Poor

24. Real-World Use Cases

Knowledge Assistant

Search and reason.

Compliance Investigator

Retrieve controls and policies.

Incident Analyst

Investigate outages.

Research Agent

Gather evidence before conclusions.

Multi-Agent Systems

Supervisor coordinating specialists.

25. Interview Questions

Q1

What is ReAct?

Q2

Why is ReAct superior to direct prompting?

Q3

How does ReAct reduce hallucinations?

Q4

What is the main production risk?

Q5

How do you govern ReAct systems?

Q6

What should be tracked operationally?

Q7

When should ReAct not be used?

Q8

How does ReAct relate to LangGraph?

26. Interview Answers

Q1

A reasoning-and-action architecture where models iteratively think, act, observe, and refine decisions.

Q2

Because the model can gather evidence rather than relying solely on internal knowledge.

Q3

By validating information through retrieval and tools before answering.

Q4

Unbounded reasoning loops and uncontrolled tool usage.

Q5

Governance layers, tool permissions, approvals, and audit trails.

Q6

Iterations, latency, success rates, tool usage, cost, and failures.

Q7

For deterministic workflows that do not require adaptive reasoning.

Q8

LangGraph provides stateful orchestration for ReAct execution.

27. Architecture Review Checklist

ReAct Readiness

- Goal defined
 - Tool selection strategy defined
 - Iteration limits configured
 - Tool permissions enforced
 - Validation implemented
 - Governance layer present
 - Observability configured
 - Cost controls configured
 - Failure recovery designed
-

28. Practical Exercises

1. Design a ReAct-based incident investigation agent.
 2. Implement a LangGraph ReAct workflow.
 3. Add governance controls to a tool-calling agent.
 4. Compare ReAct and deterministic workflows.
 5. Measure cost versus accuracy trade-offs.
-

29. Key Takeaways

- ReAct is the foundational agent execution pattern.
 - Agents alternate between reasoning and action.
 - Evidence gathering reduces hallucinations.
 - ReAct enables dynamic problem solving.
 - Tool usage transforms LLMs into agents.
 - Governance is mandatory in production.
 - LangGraph naturally implements ReAct loops.
 - ReAct is best suited for uncertain and investigative tasks.
 - Not every problem requires ReAct.
 - Most modern agent architectures are built on ReAct principles.
-

30. References

- [ReAct: Synergizing Reasoning and Acting in Language Models](#)
 - [LangGraph Documentation](#)
 - [OpenAI Function Calling Documentation](#)
 - [Anthropic Tool Use Documentation](#)
 - [CrewAI Documentation](#)
 - [AutoGen Documentation](#)
 - [Semantic Kernel Documentation](#)
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 5 – Router Pattern (Intent Classification and Dynamic Routing)